Inventory Management System for B2B SaaS

Task 1 :

Based on the code given I go through the code , I didn't know much more about the flask python but I correlate this code with my springboot code logic .

The following thinks I did noticed from this code :

1. As mentioned this is business code but while inserting / adding new product we did the commit part but if the translation fails then **Db** could not remain in consistent state .
2. If first operation of adding the new product is done but will updating the inventory did this transaction will happen or not , if it does not happen then we are not providing the facility to rollback the transaction
3. Also I have doubt when we are updating the inventory whether we are considering the existing quantity if inventory is already present
4. Also is the primary key constraints may not be handled , I think SKU is already present in database then if it is not mentioned as unique there may duplicate entries in database and while inserting or updating to inventory if may confuse about which SKU is valid one
5. At the last code is sending the string or text , in production level response code is useful also this may leads to incorrect or inaccurate consideration
6. Does anyone can add the product is there no authentication or authorization steps ?

These are some of the problems / bug I did noticed , transaction handling is one of the most or biggest bug in this code which may leads to database in inconsistent state

These fixes may reduce the dugs:

```
@RestController

@RequestMapping("/api/products")

public class ProductController {


   private final ProductRepository productRepository;

   private final InventoryRepository inventoryRepository;


   public ProductController(ProductRepository productRepository,

                    InventoryRepository inventoryRepository) {

      this.productRepository = productRepository;
```

```
    this.inventoryRepository = inventoryRepository;
}

@PostMapping
@Transactional
public ResponseEntity<?> createProduct(@RequestBody ProductRequest data) {

    //  1. Validate input
    if (data.getName() == null || data.getSku() == null || data.getPrice() == null) {
        return ResponseEntity.badRequest().body("Missing required fields");
    }

    //  2. Enforce SKU uniqueness
    if (productRepository.existsBySku(data.getSku())) {
        return ResponseEntity.badRequest().body("SKU already exists");
    }

    try {
        //  3. Create Product (NO warehouse_id here)
        Product product = new Product();
        product.setName(data.getName());
        product.setSku(data.getSku());
        product.setPrice(data.getPrice());

        productRepository.save(product);

        //  4. Create Inventory (warehouse-specific)
        int quantity = data.getInitialQuantity() != null ? data.getInitialQuantity() : 0;

        Inventory inventory = new Inventory();
        inventory.setProduct(product); // relationship mapping
```

```
            inventory.setWarehouseId(data.getWarehouseId());
            inventory.setQuantity(quantity);

            inventoryRepository.save(inventory);

            //  5. Response
            Map<String, Object> response = new HashMap<>();
            response.put("message", "Product created");
            response.put("product_id", product.getId());

            return ResponseEntity.status(HttpStatus.CREATED).body(response);

        } catch (Exception e) {
            //  Transaction will auto rollback because of @Transactional
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("Something went wrong");
        }
    }
}
```

Some more information required:

Whether the entity mentioned has all valid attributes ,

Does the important fields are missing which may helps to gather the data

Task 2 :

The database schema I did understood not fully but little , based on my assumption I did make this schema

```
-- Companies
CREATE TABLE companies (
```

```
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL
);

-- Warehouses
CREATE TABLE warehouses (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id BIGINT NOT NULL,
    name VARCHAR(255),
    location VARCHAR(255),

    FOREIGN KEY (company_id) REFERENCES companies(id)
);

-- Products
CREATE TABLE products (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    sku VARCHAR(100) NOT NULL,
    price DECIMAL(10,2),

    UNIQUE (sku), -- assuming global uniqueness (might need clarification)
    FOREIGN KEY (company_id) REFERENCES companies(id)
);

-- Inventory (product in warehouse)
CREATE TABLE inventory (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
    product_id BIGINT NOT NULL,
    warehouse_id BIGINT NOT NULL,
    quantity INT DEFAULT 0,

    UNIQUE (product_id, warehouse_id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- Inventory history (track changes)
CREATE TABLE inventory_transactions (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    product_id BIGINT,
    warehouse_id BIGINT,
    change_quantity INT,
    change_type VARCHAR(50), -- e.g., SALE, RESTOCK
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Suppliers
CREATE TABLE suppliers (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
    contact_email VARCHAR(255)
);

-- Product-Supplier mapping (many-to-many)
CREATE TABLE product_suppliers (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
    product_id BIGINT,
    supplier_id BIGINT,

    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
);

-- Product bundles (self-referencing)
CREATE TABLE product_bundles (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    parent_product_id BIGINT,
    child_product_id BIGINT,
    quantity INT,

    FOREIGN KEY (parent_product_id) REFERENCES products(id),
    FOREIGN KEY (child_product_id) REFERENCES products(id)
);
```

Assumption I assumed :

1. SKU is Unique
2. Inventory history is append only no update
3. Inventory product is belongs to one company
4. Inventory is tracked per warehouse
5. I assume bundles are the just combination of existing products

Some question to product team :

1. As for above assumption can product belongs to the multiple companies?
2. Warehouse share inventory across companies?
3. Can multiple supplier supply same product?
4. Does supplier pricing also taken into the consideration?
5. Is bundle pricing is fixed or dynamic?
6. Should the negative inventories are allowed?

Task 3 :

Here is the git hub link to view the Implemnetation

https://github.com/RiteshD11/-StockFlow---a-B2B-inventory-management-platform.git